INTELIGENCIA ARTIFICIAL: DATA MINING II

Proyecto de Algoritmos de Regresión

Datos Del Proyecto

- **Materia:** MACHINE LEARNING SUPERVISED AND UNSUPERVISED
- **Tarea:** Tarea 1
- **Docente:** Alexis Iveth Correa Vivas
- **Autores:**
- Evelin Marisol Rosero Ordoñez
- Mayra Cecilia Salazar Grandes
- Samantha Carolina Buitron Pambabay
- Jose Manuel Espinoza Bone

Este proyecto aplica dos algoritmos de regresión a un conjunto de datos usando Python. El flujo descarga el dataset, lo descomprime, lo carga con `pandas`, realiza limpieza, visualiza información relevante, divide los datos en entrenamiento/prueba, entrena los modelos y compara sus resultados.

Estructura

- `App.py`: Script principal para cargar datos, entrenar modelos y evaluar resultados.
- `Data/`: Archivos del conjunto de datos.
- `Assets/`: Imágenes, documentos o recursos del proyecto, como `Semana1.pdf`.
- `Assets/Results/`: Copia de los gráficos generados para evidenciar los resultados del análisis.
- `Base/`: Archivos locales ignorados por Git.
- `Temp/`: Carpeta temporal donde se descarga y descomprime el dataset durante la ejecución. Está ignorada por Git.
- `Results/`: Carpeta donde se guardan los gráficos generados por cada ejecución. Está ignorada por Git.
- `App.bat`: Lanzador para Windows CMD.
- `App.ps1`: Lanzador para Windows PowerShell.
- `App`: Lanzador para sistemas Unix-like.

1. Descargar Este Repositorio

Clonar el repositorio con Git:

```
git clone https://github.com/UIDE-Tareas/10-Machine-Learning-Supervised-And-Unsupervised-Tarea1.git
cd 10-Machine-Learning-Supervised-And-Unsupervised-Tarea1
```

También se puede descargar desde GitHub como archivo ZIP y extraerlo en el computador.

2. Ejecutar El Proyecto

No ejecutar `App.py` directamente. Se debe usar el lanzador correspondiente al sistema operativo.

La razón es que el lanzador crea un entorno virtual `.venv` y ejecuta `App.py` usando el Python de ese entorno. De esta forma, las librerías necesarias se instalan dentro del proyecto y no modifican el ambiente local de la máquina que ejecuta el script.

Windows CMD

```
App.bat
```

Windows PowerShell

```
.\App.ps1
```

Sistemas Unix-like

```
chmod +x App
./App
```

3. Fases Del Proyecto

El script sigue una estructura por etapas:

1. **Dataset URL**

Muestra la URL del dataset utilizado.

2. **Download Dataset**

Descarga `archive.zip` desde GitHub Raw hacia la carpeta `Temp/`. Si el archivo ya existe, no lo vuelve a descargar. El dataset original proviene de Kaggle (Narayan, s. f.).

3. **Read Dataset**

Descomprime el archivo ZIP si es necesario y lee `Student_Performance.csv` con `pandas`.

4. **Inspect DataFrame**

Muestra información inicial de `dfRaw`: estructura, nulos, duplicados, estadística descriptiva y primeras filas.

5. **Clean DataFrame**

Crea `dfClean` a partir de `dfRaw`. Aplica One-Hot Encoding, convierte columnas booleanas a `0/1`, elimina duplicados, elimina valores nulos y normaliza nombres de columnas. La conversión categórica se realiza con `pandas.get_dummies` (The pandas development team, s. f.).

6. **Target Distribution**

Grafica la distribución de la variable objetivo `PerformanceIndex` con histograma y boxplot.

7. **Train Test Split**

Divide el dataset limpio en entrenamiento y prueba usando `80%` para entrenamiento y `20%` para prueba mediante `train_test_split` (scikit-learn developers, s. f.-c).

8. **Train Correlation Matrices**

Muestra matrices de correlación sobre `X Train`: completa, fuerte y débil.

9. **Configure Regressors**

Configura los modelos de regresión:

- `PolynomialRegression`
- `RidgeRegression`

La regresión polinomial se implementa con `PolynomialFeatures` (scikit-learn developers, s. f.-b) y Ridge se implementa con `Ridge` de scikit-learn (scikit-learn developers, s. f.-a).

10. **Train And Evaluate Regressors**

Entrena los dos modelos, predice sobre test, calcula métricas y genera gráficos de resultados.

11. **Compare Regressors**

Compara los modelos usando `RMSE` y `R2`, e indica cuál tiene mejor desempeño.

4. Dataset Utilizado

El dataset usado es:

```
Student_Performance.csv
```

Fuente original:

```
https://www.kaggle.com/datasets/nikhil7280/student-performance-multiple-linear-regression
```

Este dataset se usa porque contiene una variable objetivo numérica, `PerformanceIndex`, adecuada para problemas de regresión supervisada. Las variables predictoras incluyen horas de estudio, puntajes previos, horas de sueño, exámenes practicados y actividades extracurriculares (Narayan, s. f.).

Descripción Del Dataset

El dataset **Student Performance** fue diseñado para examinar factores que pueden influir en el rendimiento académico de estudiantes. Contiene `10,000` registros, donde cada fila representa a un estudiante y contiene variables predictoras junto con un índice de rendimiento.

Variables predictoras:

- `Hours Studied`: total de horas estudiadas por cada estudiante.
- `Previous Scores`: puntajes obtenidos por los estudiantes en pruebas anteriores.

- `Extracurricular Activities`: indica si el estudiante participa en actividades extracurriculares, con valores `Yes` o `No`.
- `Sleep Hours`: promedio de horas de sueño por día.
- `Sample Question Papers Practiced`: cantidad de cuestionarios o exámenes de práctica realizados.

Variable objetivo:

- `Performance Index`: medida del rendimiento académico general del estudiante. Está redondeada al entero más cercano y toma valores entre `10` y `100`; valores más altos indican mejor rendimiento.

El objetivo del dataset es permitir el análisis de la relación entre las variables predictoras y el índice de rendimiento. En este proyecto se usa para entrenar modelos que intentan predecir `PerformanceIndex` a partir de hábitos de estudio, rendimiento previo, sueño, práctica y actividades extracurriculares.

Es importante aclarar que este dataset es **sintético** y fue creado con fines ilustrativos. Por lo tanto, las relaciones observadas entre variables no necesariamente representan escenarios reales.

Licencia: según la descripción publicada en Kaggle, cualquier persona puede compartir y usar los datos (Narayan, s. f.).

5. Archivos Generados

Durante la ejecución se generan archivos temporales y resultados:

- `Temp/archive.zip`: archivo ZIP descargado.
- `Temp/Student_Performance.csv`: archivo CSV descomprimido.
- `Results/YYYYMMdd-HHMMSS/`: carpeta de resultados de una ejecución específica.
- `Results/YYYYMMdd-HHMMSS/CorrelationMatrix_ALL.png`: matriz de correlación completa.
- `Results/YYYYMMdd-HHMMSS/CorrelationMatrix_STRONG.png`: matriz de correlación fuerte.
- `Results/YYYYMMdd-HHMMSS/CorrelationMatrix_WEAK.png`: matriz de correlación débil.
- `Results/YYYYMMdd-HHMMSS/PerformanceIndex_Distribution.png`: distribución de la variable objetivo.
- `Results/YYYYMMdd-HHMMSS/PolynomialRegression_RegressionResults.png`: gráficos del modelo polinomial.
- `Results/YYYYMMdd-HHMMSS/RidgeRegression_RegressionResults.png`: gráficos del modelo Ridge.

Las carpetas `Temp/` y `Results/` están en `.gitignore` para evitar subir archivos generados o temporales al repositorio.

Los gráficos de la ejecución revisada también se copiaron en:

```
Assets/Results/
```

Esto permite conservar evidencia visual dentro del repositorio sin depender de la carpeta temporal `Results/`.

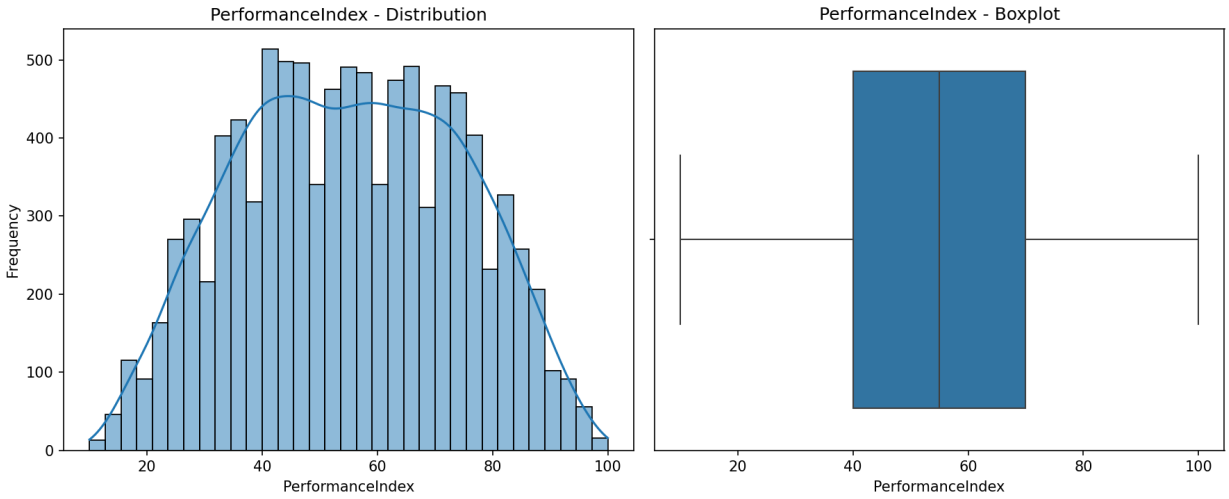
6. Análisis De Gráficos Generados

Distribución De `PerformanceIndex`

Archivo:

```
Assets/Results/PerformanceIndex_Distribution.png
```

Distribución de PerformanceIndex



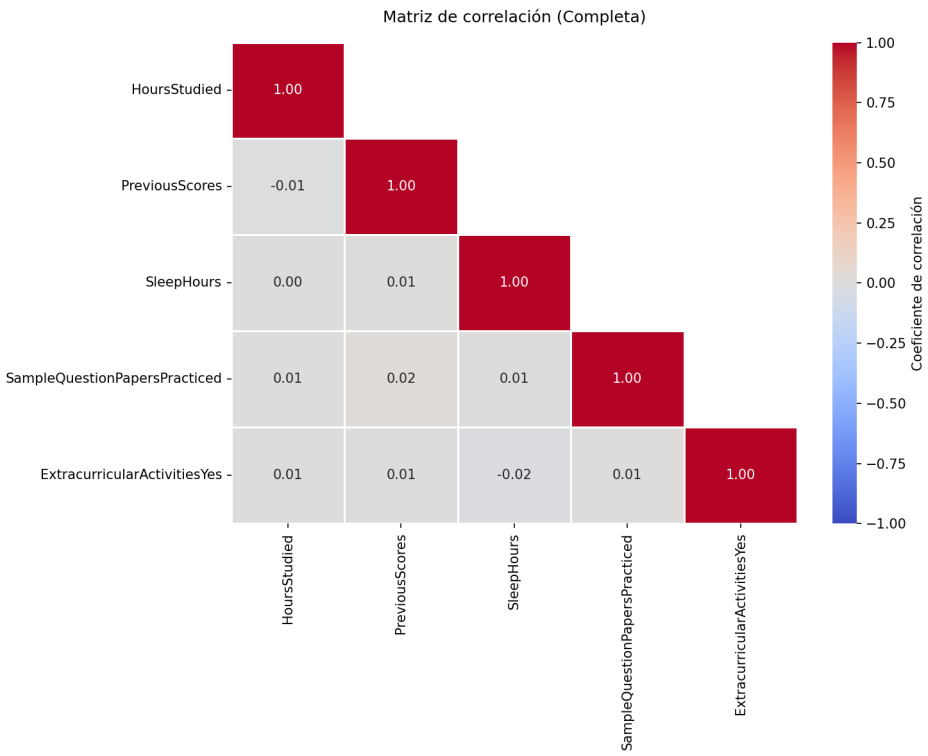
El histograma muestra que `PerformanceIndex` se distribuye en un rango aproximado de `10` a `100`, con mayor concentración en valores medios. El boxplot no evidencia valores extremos fuertes, por lo que la variable objetivo parece adecuada para modelos de regresión sin una limpieza agresiva de outliers.

Matrices De Correlación

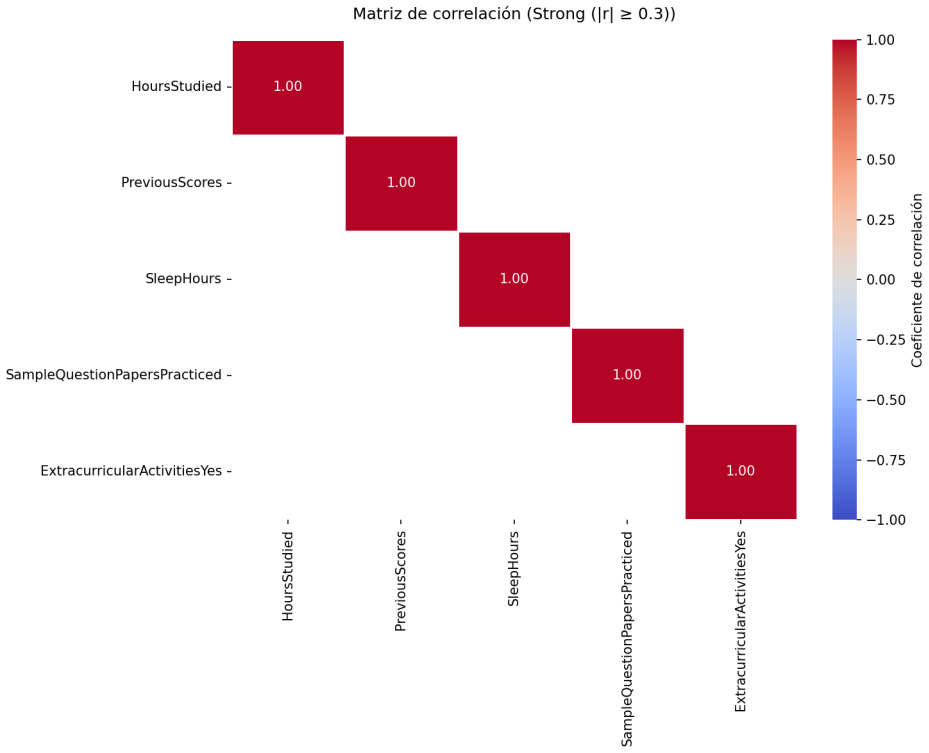
Archivos:

```
Assets/Results/CorrelationMatrix_ALL.png
Assets/Results/CorrelationMatrix_STRONG.png
Assets/Results/CorrelationMatrix_WEAK.png
```

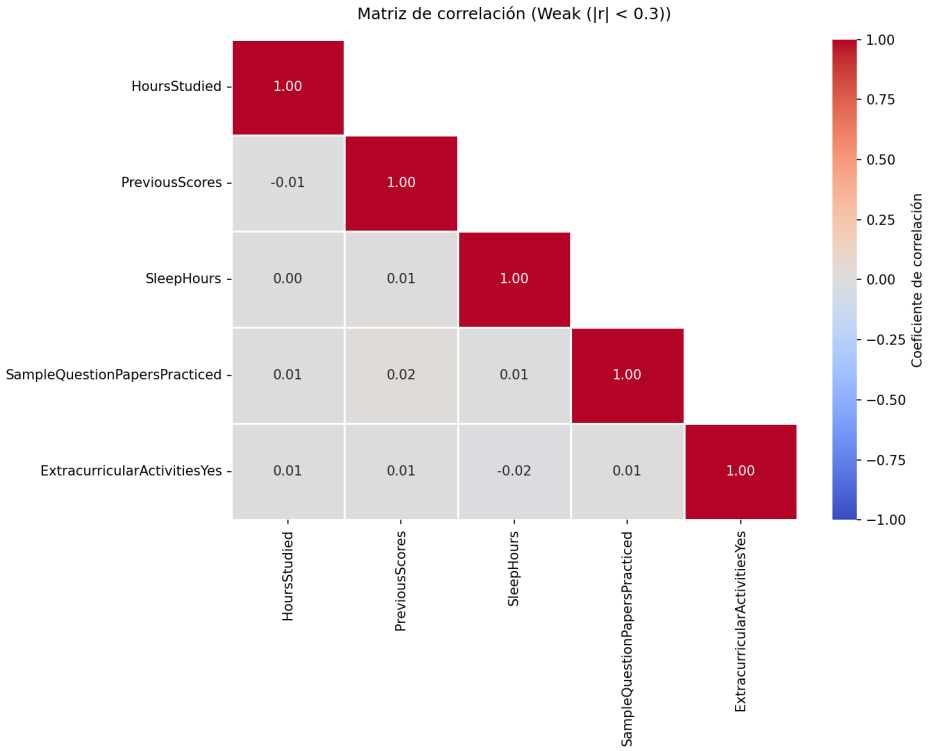
Matriz de correlación completa



Matriz de correlación fuerte



Matriz de correlación débil



La matriz completa se calcula sobre `X_Train`, es decir, solo sobre las variables predictoras del conjunto de entrenamiento. Esto evita usar información del conjunto de prueba durante el análisis exploratorio posterior al split.

En la matriz completa se observa que las correlaciones entre predictores son bajas. Con el umbral `0.30`, la matriz fuerte no muestra relaciones fuertes entre variables predictoras fuera de la diagonal. Esto indica baja multicolinealidad entre las variables de entrada, lo cual es positivo para modelos lineales y regularizados como Ridge.

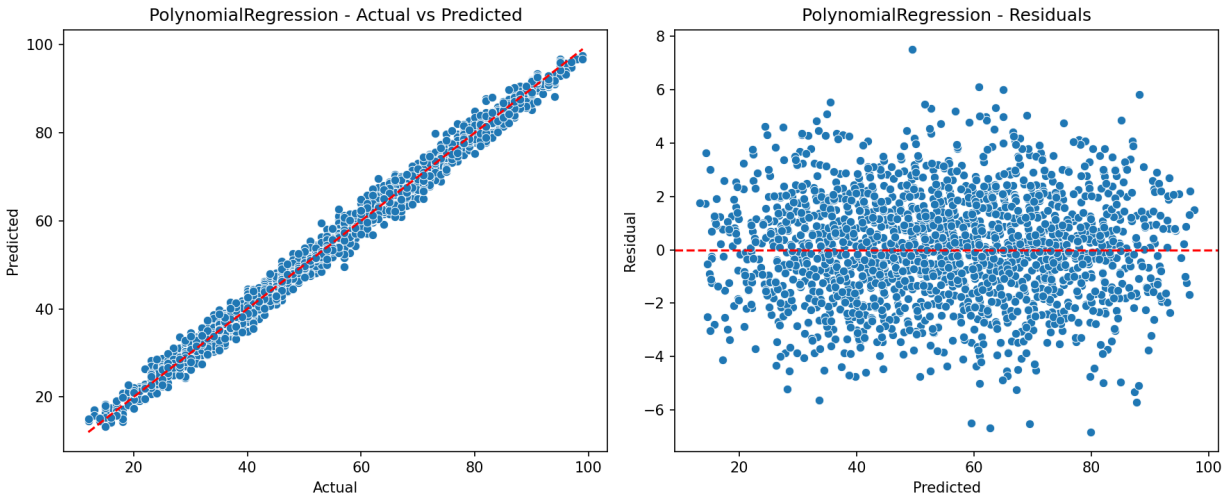
La columna `ExtracurricularActivitiesYes` aparece como variable numérica porque después del One-Hot Encoding las columnas booleanas se convierten a `0/1`.

Resultados De Regresión

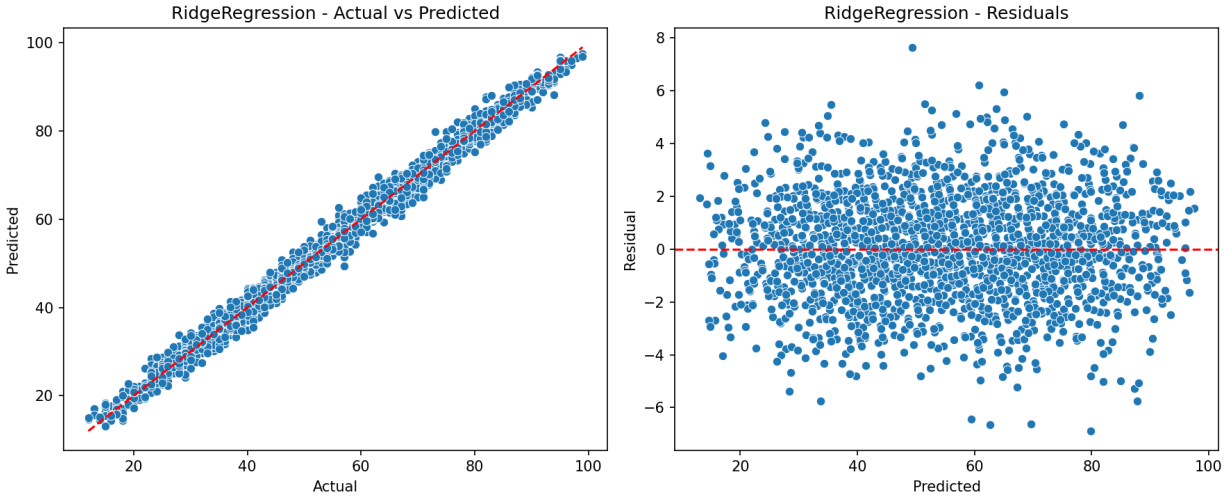
Archivos:

```
Assets/Results/PolynomialRegression_RegressionResults.png
Assets/Results/RidgeRegression_RegressionResults.png
```

Resultados de Polynomial Regression



Resultados de Ridge Regression



Los gráficos `Actual vs Predicted` muestran que las predicciones de ambos modelos se alinean muy cerca de la línea ideal. Esto sugiere que los modelos capturan bien la relación entre las variables predictoras y `PerformanceIndex`.

Los gráficos de residuos muestran errores distribuidos alrededor de `0`, sin un patrón curvo evidente. Esto es una buena señal: indica que los modelos no presentan un sesgo visual fuerte en las predicciones. Ridge y Polynomial Regression muestran resultados visualmente muy similares en esta ejecución.

7. Constantes Configurables

Las principales constantes configurables están al inicio de `App.py`:

- `RANDOM_STATE = 216`

Controla la reproducibilidad de operaciones aleatorias como el split.

- `TRAIN_RATIO = 0.80`

Define el porcentaje de datos usado para entrenamiento.

- `TEST_RATIO = 0.20`

Define el porcentaje de datos usado para prueba.

- `POLYNOMIAL_DEGREE = 2`

Define el grado usado por la regresión polinomial.

- `RIDGE_ALPHA = 1.0`

Define la fuerza de regularización del modelo Ridge.

- `CORRELATION_THRESHOLD = 0.30`

Define el umbral usado para clasificar correlaciones fuertes o débiles.

- `SHOW_PLOTS = True`

Controla si los gráficos se muestran en pantalla.

- `SAVE_PLOTS = True`

Controla si los gráficos se guardan en la carpeta `Results/`.

- `DATASET_URL`

URL de la página original del dataset en Kaggle.

- `DATASET_ARCHIVE_RAW_URL`

URL directa del archivo ZIP en GitHub Raw.

- `DATASET_ARCHIVE_FILENAME = "archive.zip"`

Nombre del archivo ZIP descargado.

- `DATASET_CSV_FILENAME = "Student_Performance.csv"`

Nombre del CSV dentro del ZIP.

- `TARGET_COLUMN = "PerformanceIndex"`

Variable objetivo que los modelos intentan predecir.

- `TEMP_DIR = "Temp"`

Carpeta temporal usada para descarga y descompresión.

- `RESULTS_DIR = Path("Results")`

Carpeta principal donde se almacenan resultados.

8. Modelos Y Métricas

Los modelos utilizados son:

- `PolynomialRegression`: usa `PolynomialFeatures`, `StandardScaler` y `LinearRegression`.
- `RidgeRegression`: usa `StandardScaler` y `Ridge`.

Polynomial Regression

La regresión polinomial se usa cuando se quiere permitir que el modelo capture relaciones un poco más flexibles que una línea recta. En lugar de trabajar solo con las columnas originales, primero se crean nuevas columnas a partir de ellas, por ejemplo cuadrados o combinaciones entre variables.

En este proyecto, `PolynomialFeatures` crea esas nuevas columnas (scikit-learn developers, s. f.-b). Después, `StandardScaler` ajusta la escala de los datos para que las columnas sean comparables entre sí. Finalmente, `LinearRegression` aprende la relación entre esas columnas transformadas y `PerformanceIndex`.

Aunque aparece `LinearRegression`, el modelo sigue siendo una regresión polinomial porque la regresión lineal se aplica después de transformar los datos. En otras palabras, no se entrena sobre los datos originales tal como vienen, sino sobre una versión enriquecida con características polinomiales.

Se usa `POLYNOMIAL_DEGREE = 2` porque permite capturar relaciones simples no lineales sin volver el modelo demasiado complejo. Un grado más alto podría crear muchas más columnas y aumentar el riesgo de que el modelo aprenda demasiado bien el entrenamiento, pero generalice peor en datos nuevos.

Ridge Regression

Ridge Regression es una regresión lineal con regularización (scikit-learn developers, s. f.-a). Esto significa que, además de buscar buenas predicciones, el modelo intenta evitar coeficientes demasiado grandes. Esa restricción ayuda a que el modelo sea más estable.

Este modelo es útil cuando se quiere una alternativa más controlada que una regresión lineal tradicional. También funciona bien cuando hay varias variables predictoras y se desea reducir el riesgo de que una variable domine demasiado el modelo.

En este proyecto se usa junto con `StandardScaler` porque Ridge trabaja mejor cuando las variables están en escalas comparables. Se configura `RIDGE_ALPHA = 1.0` como valor inicial. Un valor mayor aumenta la regularización; un valor menor hace que el modelo se parezca más a una regresión lineal común.

Las métricas calculadas son:

- `MAE`: indica el error promedio en unidades de `PerformanceIndex`, sin considerar la dirección del error.
- `MSE`: penaliza más los errores grandes porque eleva los errores al cuadrado.
- `RMSE`: es similar al `MSE`, pero vuelve a la escala original de `PerformanceIndex`, por lo que es más fácil de interpretar.
- `R2`: indica qué tanto de la variación de la variable objetivo logra explicar el modelo. Mientras más cercano a `1`, mejor.

9. Conclusión Y Recomendación

El proyecto implementa un flujo completo de regresión supervisada: descarga del dataset, carga, inspección, limpieza, codificación de variables categóricas, división train/test, visualización, entrenamiento, evaluación y comparación de modelos.

El dataset es adecuado para este ejercicio porque tiene una variable objetivo numérica (`PerformanceIndex`) y varias variables predictoras relacionadas con hábitos o antecedentes académicos. Sin embargo, al ser un dataset sintético, los resultados deben entenderse como parte de una práctica académica. No deben interpretarse como conclusiones reales sobre estudiantes.

Los gráficos de distribución muestran que `PerformanceIndex` tiene un rango amplio y sin valores extremos demasiado marcados. Las matrices de correlación indican que las variables predictoras no están fuertemente correlacionadas entre sí. Esto es positivo, porque reduce el riesgo de redundancia entre columnas.

Los gráficos `Actual vs Predicted` muestran que ambos modelos predicen valores muy cercanos a los reales. Además, los residuos se mantienen alrededor de cero, lo que sugiere que los errores no tienen un patrón fuerte evidente.

Como recomendación general, Ridge Regression es una buena opción base porque es más simple y estable. Polynomial Regression también funciona bien y puede capturar relaciones más complejas, pero debe usarse con cuidado porque puede crecer en complejidad rápidamente.

Para una mejora futura, sería recomendable probar validación cruzada y búsqueda de hiperparámetros para `RIDGE_ALPHA` y `POLYNOMIAL_DEGREE`, en lugar de dejar esos valores fijos.

10. Referencias

Narayan, N. (s. f.). *Student Performance (Multiple Linear Regression)* [Conjunto de datos]. Kaggle. Recuperado el 22 de abril de 2026, de https://www.kaggle.com/datasets/nikhil7280/student-performance-multiple-linear-regression

scikit-learn developers. (s. f.-a). *Ridge*. scikit-learn. Recuperado el 22 de abril de 2026, de https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html

scikit-learn developers. (s. f.-b). *PolynomialFeatures*. scikit-learn. Recuperado el 22 de abril de 2026, de https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html

scikit-learn developers. (s. f.-c). *train_test_split*. scikit-learn. Recuperado el 22 de abril de 2026, de https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

The pandas development team. (s. f.). *pandas.get_dummies*. pandas. Recuperado el 22 de abril de 2026, de https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html